EX. NO. : NAME :
DATE: REG. NO. :

SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
Category - I University by UGC
Accredited "A++" by NAAC | Approved by AICTE
www.sathyabama.ac.in

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

SI3BLH42 DIGITAL SIGNAL PROCESSING LAB

EX. NO. : NAME :
DATE: REG. NO. :

LIST OF EXPERIMENT

Sl. No.	DESCRIPTION	PAGE NO.
1.	Generation of signals- Discrete form & Continuous form	2
2.	Periodic & Aperiodic sequences	
3.	Linear Convolution	5
	Circular Convolution	6
4.	Auto Correlation	8
	Cross Correlation	9
5.	Discrete Fourier Transform	10
	Inverse Discrete Fourier Transform	11
6.	a.Butterworth Low Pass Filter b.Butterworth High Pass Filter c.Butterworth Band Pass Filter d. Butterworth Band stop Filter	12
7.	a. Chebyshev Low Pass Filter b. Chebyshev High Pass Filter c. Chebyshev Band pass Filter d. Chebyshev Band stop Filter	16
8.	Rectangular & Hamming Window	22
	Kaiser Window	23
9.	a) Z Transform b) Inverse z transform	28
10.	a) Study Of TMS320C3X Processor b) Study Of ADSP21XX Processor c) Study Of ADSP21XX Processor	30

EX. NO. : NAME :
DATE: REG. NO. :

GENERATION OF SIGNALS
DISCRETE AND CONTINUOUS FORM

AIM:

To write a program in MATLAB to generate signals in discrete and continuous form.

ALGORITHM:

1. Get the value of n for the sequences.
2. Use the desired formula for generating the signals.
3. Use the keyword stem for generating the discrete signal.
4. Use the keyword plot for generating the continuous signal.
5. Plot the graph.
6. Stop.

FORMULA USED:

Unit impulse: $$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

Unit step: $$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

Sine signal: $x(n)= A \sin (w_n+\theta)$

Cosine signal: $x(n)= A \cos (w_n+\theta)$

Exponential Signal: $x(n)= \exp (n)$

EX. NO. : NAME :
DATE: REG. NO. :

PROGRAM :

```
clc;
N=21;
x=ones(1,N);
n=0:1:N-1;
subplot(6,2,1);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Discrete Unit Step Signal');

N=21;
x=ones(1,N);
n=0:1:N-1;
subplot(6,2,2);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('Continuous Unit Step Signal');

N=21;
x=sin(.2*pi*n);
n=0:1:N-1;
subplot(6,2,3);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Discrete Sinusoidal Signal');

N=21;
x=sin(.2*pi*n);
n=0:1:N-1;
subplot(6,2,4);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('continuous sinusoidal signal');

N=21;
x=cos(.2*pi*n);
n=0:1:N-1;
```

```
subplot(6,2,5);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Discrete Cosine Signal');

N=21;
x=cos(.2*pi*n);
n=0:1:N-1;
subplot(6,2,6);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('continuous cosine Signal');

N=21;
x=n;
n=0:1:N-1;
subplot(6,2,7);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Discrete Ramp Signal');

N=21;
x=n;
n=0:1:N-1;
subplot(6,2,8);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('continuous ramp signal');

N=21;
x=exp(n);
n=0:1:N-1;
subplot(6,2,9);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Discrete Exponential Signal');

N=21;
```

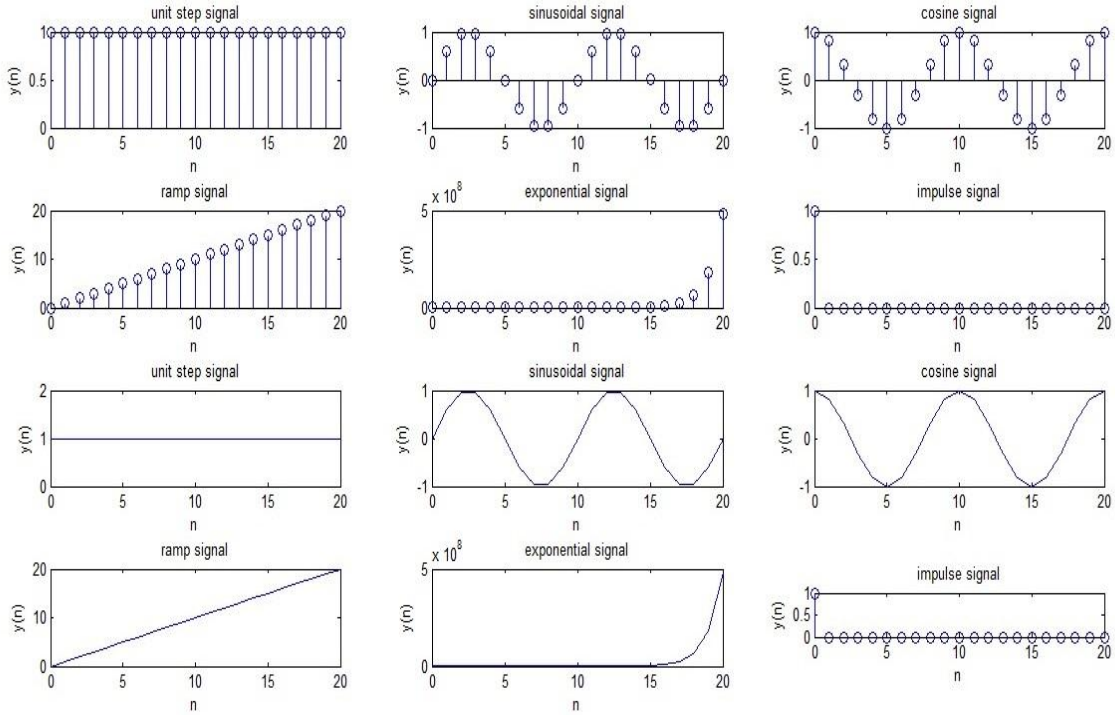
EX. NO. : NAME :
DATE: REG. NO. :

```
x=exp(n);
n=0:1:N-1;
subplot(6,2,10);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('Continuous Exponential Signal');

N=21;
x=[ones(1,1),zeros(1,N-1)];
n=0:1:N-1;
subplot(6,2,11);
stem(n,x);
xlabel('n');


N=21;
x=[ones(1,1),zeros(1,N-1)];
n=0:1:N-1;
subplot(6,2,12);
plot(n,x);
xlabel('n');
ylabel('y(n)');
title('Continuous Impulse Signal');
```

EX. NO. : NAME :

DATE: REG. NO. :

SIMULATED OUTPUT:



RESULT:

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

LINEAR CONVOLUTION

AIM:

To write a program in MATLAB to perform linear convolution of two signals.

ALGORITHM:

1. Get two signals g(m) and h(p) in matrix form.
2. The convoluted signal is denoted as y(n).
3. y(n) is given by the formula

$$y(n) = \sum_{k=-\infty}^{\infty} [\, g(k)\, h(n-k) \,] \quad \text{where } n=0 \text{ to } m+p-1$$

4. Stop.

FORMULA USED :

The linear convolution is given by

$$g(n) = \sum_{k=-\infty}^{\infty} h(k) f(n-k)$$

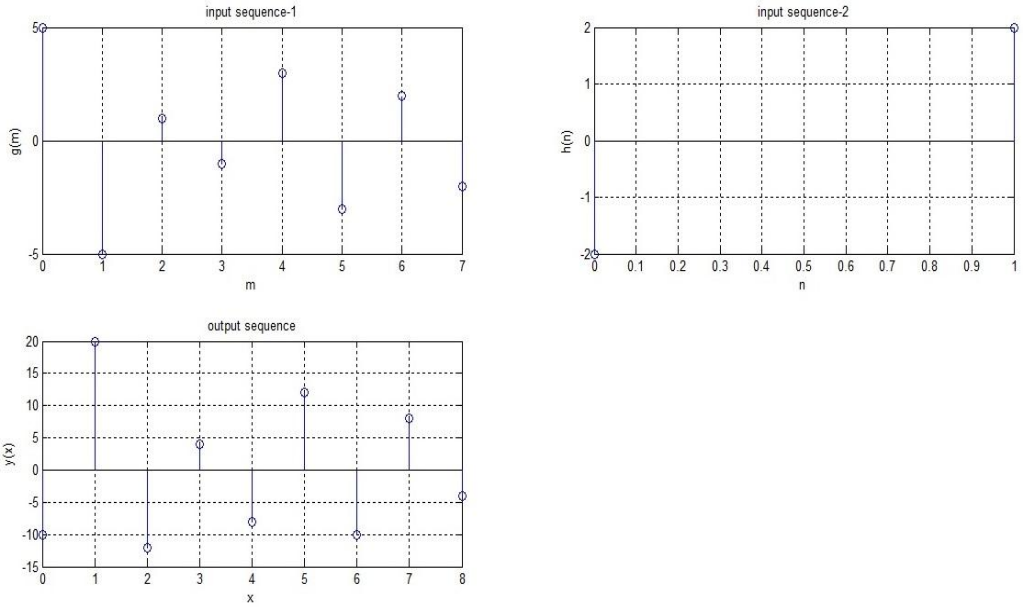
PROGRAM:-

```
close all;
clear all;
clc;
g=[1,-1,2,-2,3,-3,4,-4];
h=[-1,1];
n1=length(g);
n2=length(h);
m=0:1:n1-1;
figure();
subplot(2,2,1);
stem(m,g);
xlabel('m');
ylabel('g(m)');
grid;
title('Input Sequence-1');

n=0:1:n2-1;
subplot(2,2,2);
stem(n,h);
xlabel('n');
ylabel('h(n)');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
grid;
title('Input Sequence-2');

y=conv(g,h);
n3=n1+n2-1;
x=0:1:n3-1;
subplot(2,2,3);
stem(x,y);
disp(y);
xlabel('x');
ylabel('y(x)');
grid;
title('Output Sequence');
```

SIMULATED OUTPUT:



RESULT:

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

CIRCULAR CONVOLUTION

AIM :

To write a program in MATLAB to perform circular convolution of two signals.

ALGORITHM :

1. Get the two input sequences g(m) and h(p).
2. Get the length of the sequences.
3. check whether the lengths are equal using the loop
4. if(N3=0)
   h=[h,zeros (1,N3)];
   else
   g=[g,zeros (1,-N3)];
   end
5. Compute the circular convolution of the sequence.
6. y(n) = y(n)+g(i)*h(j);
7. stop.

FORMULA USED:

Circular Convolution $x_3(n) = x_1(n) \circledN x_2(n)$

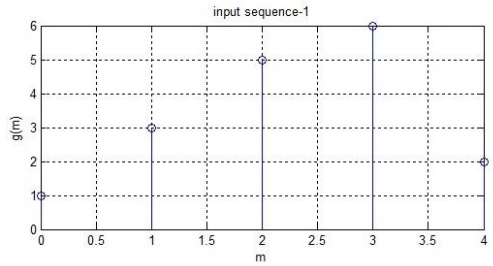
PROGRAM :

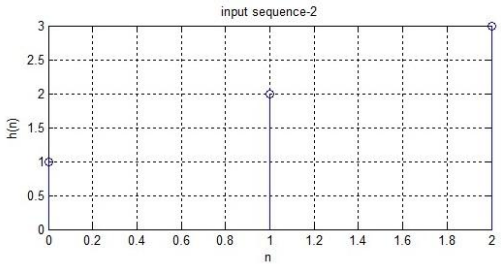
```
close all;
clear all;
clc;
g=[1,3,5,6,2];
h=[1,2,3];
n1=length(g);
n2=length(h);
m=0:1:n1-1;
subplot(2,2,1);
stem(m,g);
xlabel('m');
ylabel('g(m)');
grid;
title('Input Sequence-1');
```

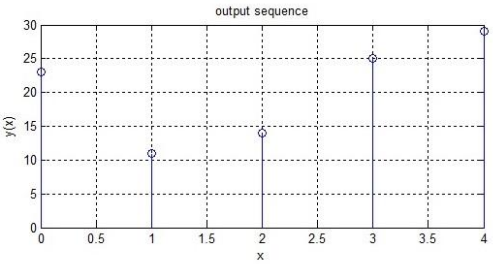
```
n=0:1:n2-1;
subplot(2,2,2);
stem(n,h);
xlabel('n');
ylabel('h(n)');
grid;
title('Input Sequence-2');

N=max(n1,n2);
n3=n1-n2;
if(n3>0)
   h=[h,zeros(1,n3)];
else
   g=[g,zeros(1,-n3)];
end
for x=1:N
   y(x)=0;
   for i=1:N
      j=x-i+1;
      if(j<=0)
         j=N+j;
      end
      y(x)=y(x)+g(i)*h(j);
   end
end
disp(y);
x=0:1:N-1;
subplot(2,2,3);
stem(x,y);
xlabel('x');
ylabel('y(x)');
grid;
title('Output Sequence');
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT :







RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

AUTO – CORRELATION

AIM :

To write a program in MATLAB to perform auto correlation of an input sequence.

ALGORITHM :

1. Get the signal x(n) of length n in matrix form.
2. The correlated signal is denoted as y(n).
3. y(n) is given by the formula

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)\, x(k-n)$$

where n= -(N-1) to (N-1).

4. Stop.

FORMULA USED:

Autocorrelation of $g[n]$:

$$r_{gg}[\ell] = \sum_{n=-\infty}^{\infty} g[n]g[n-\ell] = g[n] \circledast g[-n]$$

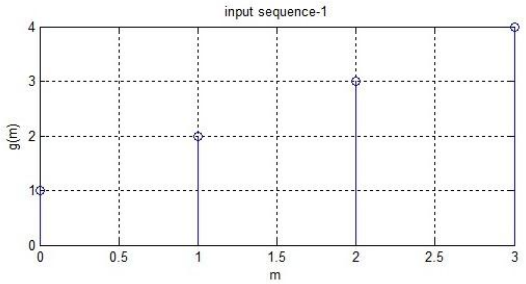
PROGRAM:

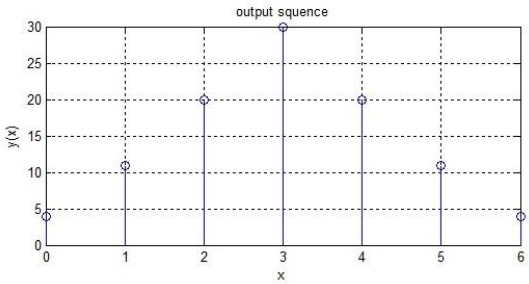
```
close all;
clear all;
clc;
 g=[1,2,3,4];
n1=length(g);
m=0:1:n1-1;
figure();
subplot(2,2,1);
stem(m,g);
xlabel('m');
ylabel('g(m)');
grid();
title('Input SEquence-1');

y=xcorr(g,g);
n2=n1+n1-1;
x=0:1:n2-1;
subplot(2,2,2);
stem(x,y);
```

EX. NO. : NAME :
DATE: REG. NO. :

```
disp(y);
xlabel('x');
ylabel('y(x)');
grid;
title('Output Sequence');
```

SIMULATED OUTPUT :





RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

CROSS – CORRELATION

AIM :

To write a program in MATLAB to perform cross correlation of two sequences.

ALGORITHM :

1. Get two signals x(m) & h(p) in the matrix form.
2. The correlated signal is denoted as y(n).
3. y(n) is given by the formula

$$y(n)= \sum_{k=-\infty}^{\infty} [\, x(k)\, h(k-n)]$$

where n=-[max(m,p)-1] to [max(m,p)-1]

4. Stop.

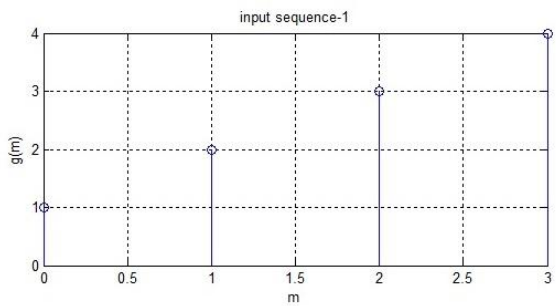
PROGRAM :

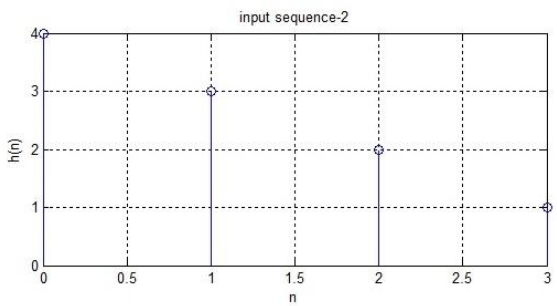
```
close all;
clear all;
clc;
g=[1,2,3,4];
h=[4,3,2,1];
n1=length(g);
n2=length(h);
m=0:1:n1-1;
figure();
subplot(2,2,1);
stem(m,g);
xlabel('m');
ylabel('g(m)');
grid;
title('Input Sequence-1');

n=0:1:n2-1;
subplot(2,2,2);
stem(n,h);
xlabel('n');
ylabel('h(n)');
grid;
title('Input Sequence-2');
```

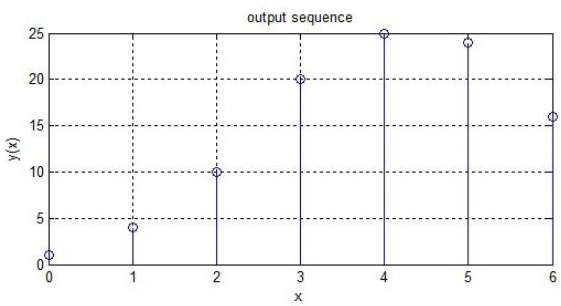
EX. NO. : NAME :
DATE: REG. NO. :

```
y=xcorr(g,h);
n3=n1+n2-1;
x=0:1:n3-1;
subplot(2,2,3);
stem(x,y);
disp(y);
xlabel('x');
ylabel('y(x)');
grid;
title('Output Sequence');
```

SIMULATED OUTPUT :







RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

DISCRETE FOURIER TRANSFORM

AIM :

To write a program in MATLAB to perform DFT of an input sequence.

ALGORITHM :

1. Get the signal x(n) of length N in matrix form.
2. Get the N value.
3. The transformed signal is denoted as

$$x(k) = \sum_{n=0}^{N-1} x(n)\, e^{((-j*2pi*n*k)/N)} \qquad \text{for } k=0 \text{ to } (N-1).$$

4. Stop.

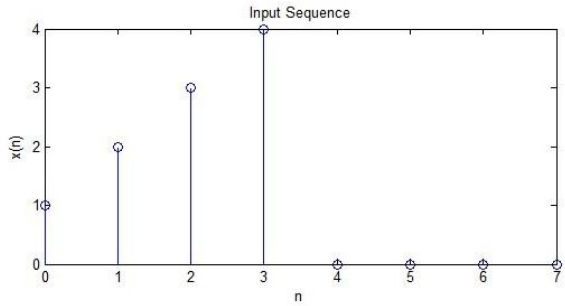
PROGRAM :

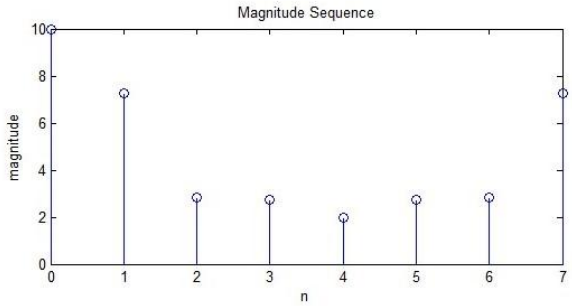
```
close all;
clear all;
clc;
x=[1,2,3,4,0,0,0,0];
N=8;
y=fft(x,N);
mag=abs(y);
phase=angle(y);
n=0:1:N-1;
subplot(2,2,1);
stem(n,x);
xlabel('n');
ylabel('x(n)');
title('Input Sequence');

subplot(2,2,2);
stem(n,mag);
xlabel('n');
ylabel('magnitude');
title('Magnitude Sequence');

subplot(2,2,3);
stem(n,phase);
xlabel('n');
```

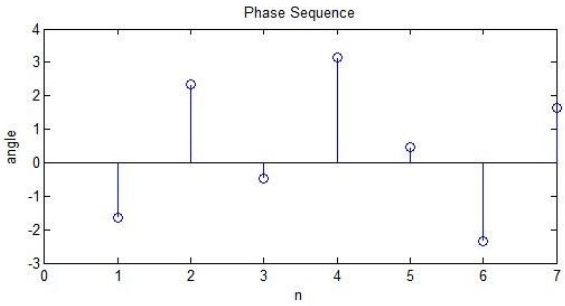
EX. NO. : NAME :
DATE: REG. NO. :

```
ylabel('angle');
title('Phase Sequence');
```

SIMULATED OUTPUT :







RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

INVERSE DISCRETE FOURIER TRANSFORM

AIM :

To write a program in MATLAB to perform IDFT of a sequence.

ALGORITHM :

1. Get the signal x(n) of length N in matrix form.
2. Get the N value.
3. The transformed signal is denoted as

   N-1

   x(n) =(1/N) **Σ** x(k) e^((-j*2pi*n*k)/N) for n=0 to (N-1).

   n=0
4. Stop.

FORMULA:

$$F[n] = \sum_{k=0}^{N-1} f[k] e^{-j\frac{2\pi}{N}nk}$$

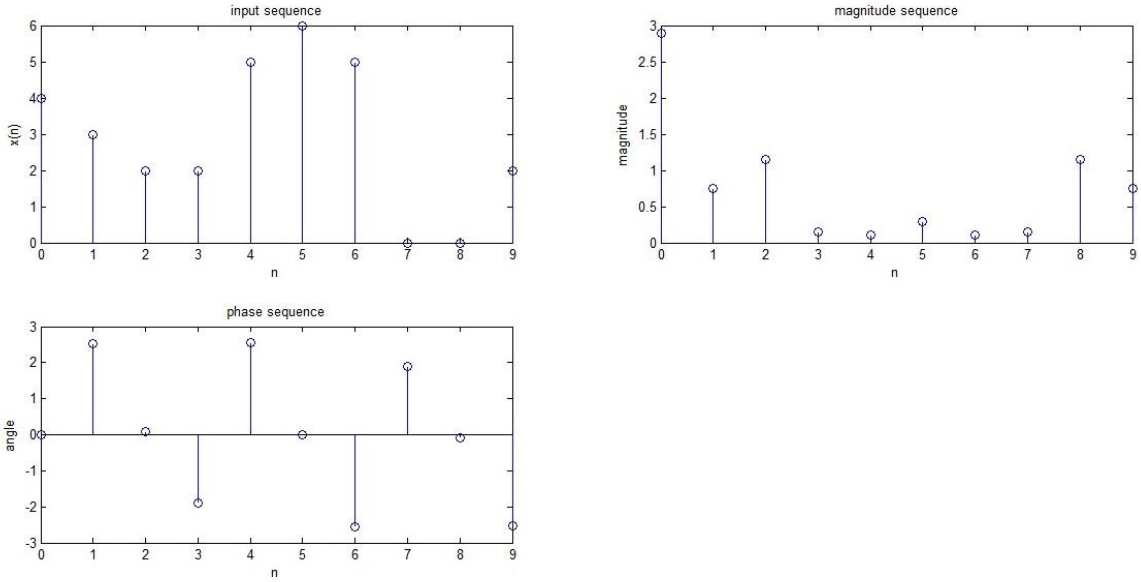
PROGRAM :

```
close all;
clear all;
clc;
x=[4,3,2,1,0,0,0,0];
N=8;
y=ifft(x,N);
mag=abs(y);
phase=angle(y);
k=0:1:N-1;
subplot(2,2,1);
stem(k,x);
xlabel('k');
ylabel('x(k)');
title('Input Sequence');

subplot(2,2,2);
stem(k,mag);
xlabel('k');
ylabel('magnitude');
title('Magnitude Sequence');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
subplot(2,2,3);
stem(n,phase);
xlabel('k');
ylabel('angle');
title('Phase Sequence');
disp(y);
```

SIMULATED OUTPUT :



RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

BUTTERWORTH LOW PASS FILTER

AIM :

To write a program in MATLAB to perform Butterworth LPF Operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Get the pass band and stop band edge frequencies.
3. Get the sampling frequency.
4. Calculate the order of the filter using the formula,

   N =(log√(10^0.1αs-1)(10^0.1αp-1))/(log(Ωs/Ωp)).

5. Find the filter coefficients.
6. Draw the magnitude and phase responses.
7. Stop.

FORMULA:

The transfer function of the Butterworth reference analog prototype filter is expressed as follows:

$$H_a(s) = \frac{1}{\prod_{k=0}^{N-1}(s - s_k)}$$

Where, Sk is the k-th pole of the Butterworth filter transfer function, N is the filter order.

$$s_k = e^{j\pi\left(\frac{1}{2}+\frac{2\cdot k+1}{2\cdot N}\right)} \text{ ; k=0, 1, ... , 2N-1}$$

For N=5, the transfer function is:

$$H_a(s) = \frac{1}{(s-s_0)\cdot(s-s_1)\cdot(s-s_2)\cdot(s-s_3)\cdot(s-s_4)} \qquad s \rightarrow \frac{\Omega_c}{s}$$

$$H_a(s) = \frac{1}{(s+0.309-j0.951)\cdot(s+0.809-j0.588)\cdot(s+1)\cdot(s+0.809+j0.588)\cdot(s+0.309+j0.951)}$$

The transformation to a high-pass analog filter:

$$s \longrightarrow \frac{s^2+\Omega_l\Omega_u}{s(\Omega_u-\Omega_l)} \qquad \text{(lowpass to bandpass)}$$

$$s \longrightarrow \Omega_p\frac{s^2+\Omega_l\Omega_u}{s(\Omega_u-\Omega_l)} \qquad \text{(lowpass to bandpass)}$$

Ω_l = lower band edge frequency

Ω_u = upper band edge frequency

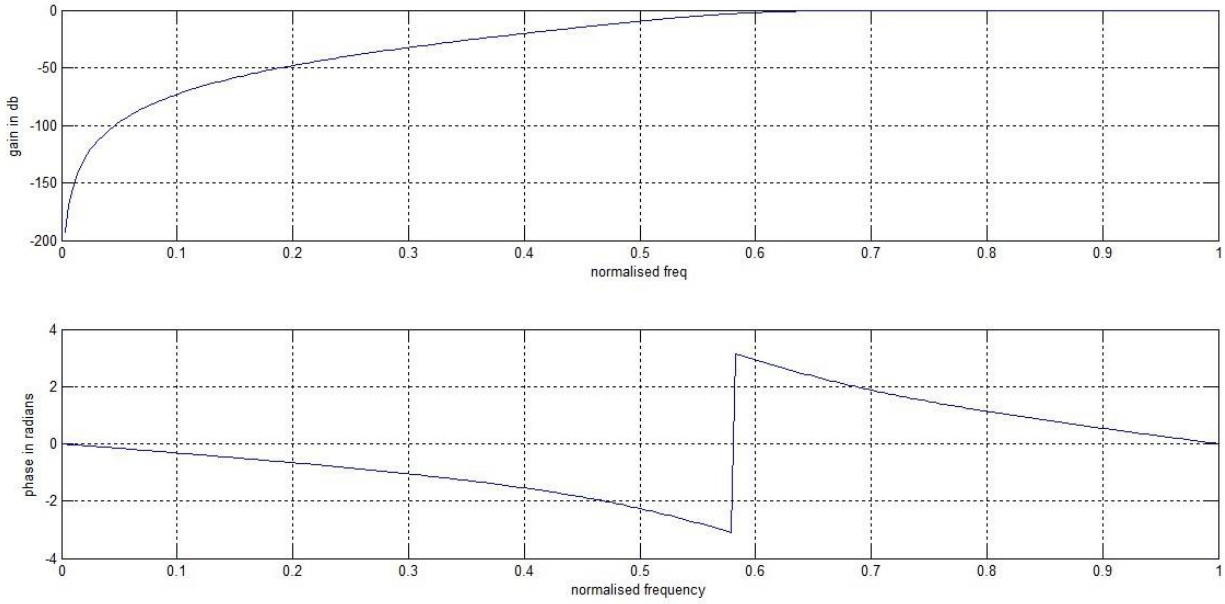
EX. NO. : NAME :
DATE: REG. NO. :

PROGRAM :

```
clc;
clear all;
close all;
alphap=input('Enter The Pass Band Ripple');
alphas=input('Enter The Stop Band Ripple');
fp=input('Enter The Pass Band Frequency');
fs=input('Enter The Stop Band Frequency');
f=input('Enter The Sampling Frequency');

omp=2*(fp/f);
oms=2*(fs/f);
[h,wn]=buttord(omp,oms,alphap,alphas)
[b,a]=butter(h,wn,'low')
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain in db');
xlabel('Normalised Frequency');

subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
xlabel('Normalized Frequency');
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT (LPF):



INPUT :

Enter the pass band ripple: 0.4
Enter the stop band ripple: 30
Enter the pass band frequency: 400
Enter the stop band frequency: 800
Enter the sampling frequency: 2000

COMMAND WINDOW:

h =4
wn =0.5821
b =0.1518 0.6073 0.9109 0.6073 0.1518
a =1.0000 0.6418 0.6165 0.1449 0.0259

RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

BUTTERWORTH HIGH PASS FILTER

AIM:

To write a program in MATLAB to perform Butterworth HPF operation.

ALGORITHM:

1. Get the pass band and stop band ripples.
2. Get the pass band and stop band edge frequencies.
3. Get the sampling frequency.
4. Calculate the order of the filter using the formula,

   N =(log√(10^0.1αs-1)(10^0.1αp-1))/(log(Ωs/Ωp)).

5. Find the filter coefficients.
6. Draw the magnitude and phase responses.
7. Stop.

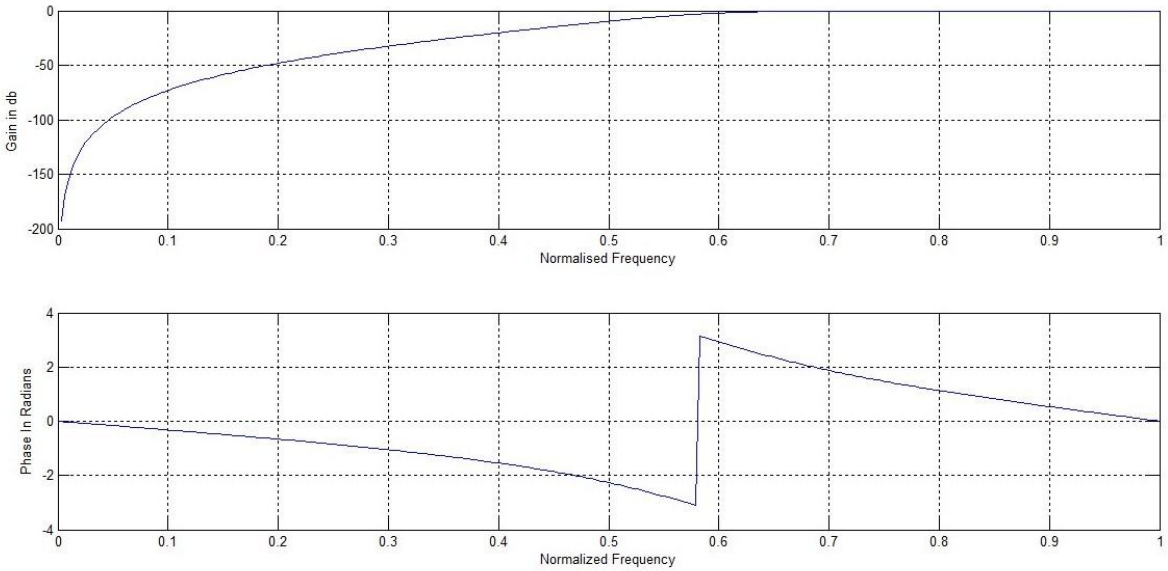
PROGRAM :

```
clc;
clear all;
close all;
alphap=input('Enter The Pass Band Ripple');
alphas=input('Enter The Stop Band Ripple');
fp=input('Enter The Pass Band Frequency');
fs=input('Enter The Stop Band Frequency');
f=input('Enter The Sampling Frequency');

omp=2*pi*(fp/f);
oms=2*pi*(fs/f);
[h,wn]=buttord(omp,oms,alphap,alphas)
[b,a]=butter(h,wn,'high')
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain in db');
xlabel('Normalised Frequency');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
xlabel('Normalized Frequency');
```

SIMULATED OUTPUT (HPF):



INPUT:

Enter the pass band ripple :0.4
Enter the stop band ripple :30
Enter the pass band frequency : 400
Enter the stop band frequency : 800
Enter the sampling frequency : 2000

COMMAND WINDOW :

h =4
wn =0.5821
b =0.0535 -0.2139 0.3209 -0.2139 0.0535
a =1.0000 0.6418 0.6165 0.1449 0.0259

RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

BUTTERWORTH BAND PASS FILTER

AIM :

To write a program in MATLAB to perform Butterworth BPF operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Calculate the order of the filter using the formula,

   N =(log√(10^0.1αs-1)(10^0.1αp-1))/(log(Ωs/Ωp)).

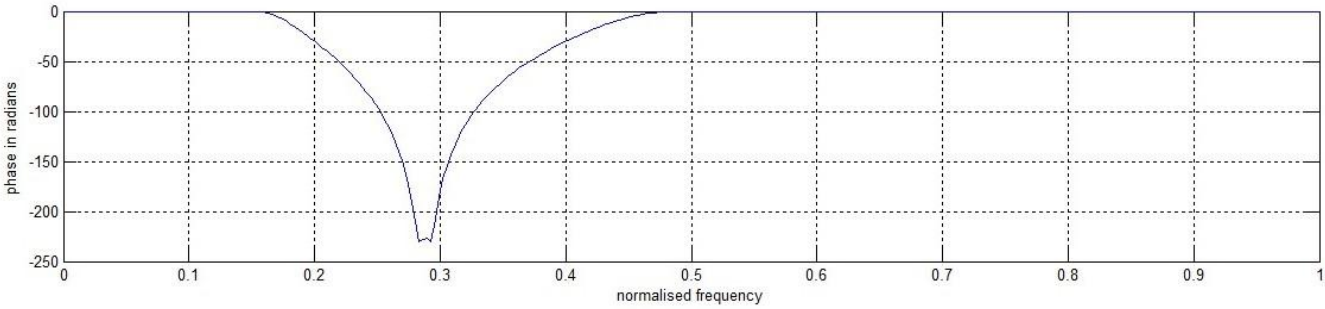
3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

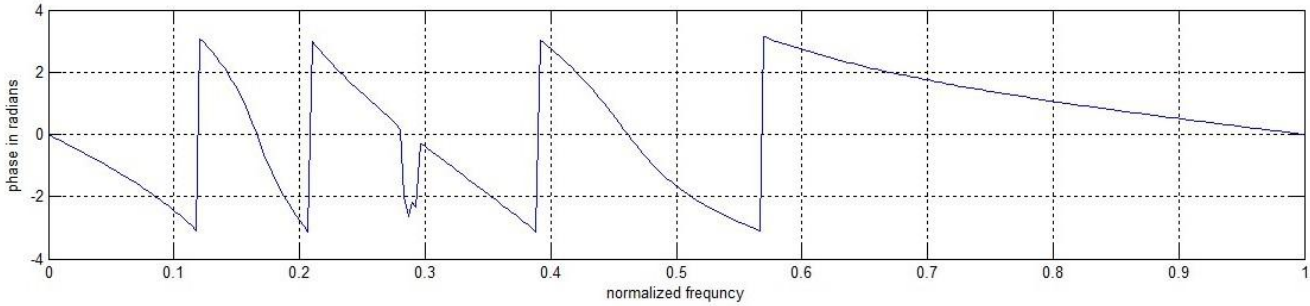
PROGRAM :

```
close all;
clear all;
clc;
alphap=input('Enter The Pass Band Ripple');
alphas=input('Enter The Stop Band Ripple');
wp=[0.2, 0.4];
ws=[0.1, 0.5];
[h,wn]=buttord(wp,ws,alphap,alphas)
[b,a]=butter(h,wn)
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain in db');
xlabel('Normalised Frequency');

subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
xlabel('Normalized Frequency');
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT:





INPUT :

Enter the pass band ripple : 0.4
Enter the stop band ripple : 30

RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

BUTTERWORTH BAND STOP FILTER

AIM:

To write a program in MATLAB to perform Butterworth BSF operation.

ALGORITHM:

1. Get the pass band and stop band ripple.
2. Calculate the order of the filter using the formula,

   N =(log√(10^0.1αs-1)/(10^0.1αp-1))/(log(Ωs/Ωp)).

3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

PROGRAM :

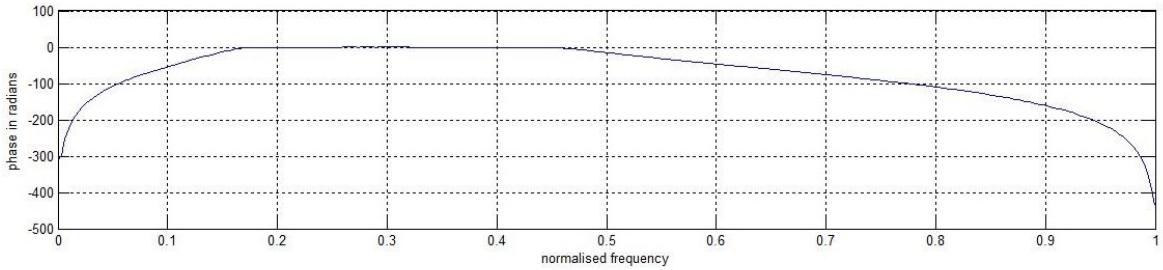
```
close all;
clear all;
clc;
alphap=input('Enter the pass band ripple');
alphas=input('Enter the stop band ripple');

wp=[0.2, 0.4];
ws=[0.1, 0.5];
[h,wn]=buttord(wp,ws,alphap,alphas)
[b,a]=butter(h,wn,'stop')
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain in db');
xlabel('Normalised Frequency');

subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase in Radians');
xlabel('Normalized Frequency');
```

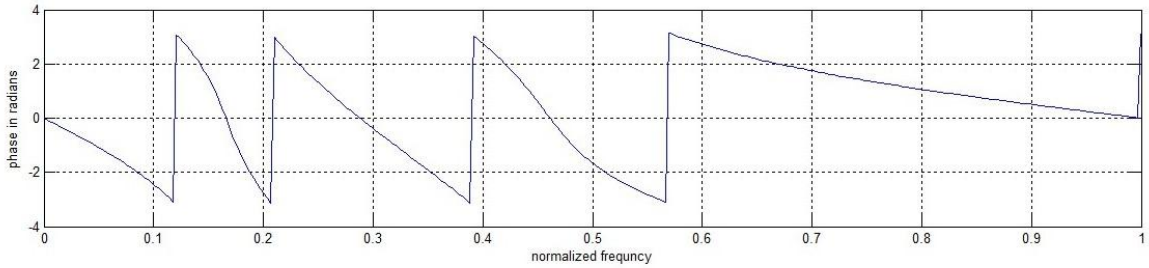
EX. NO. : NAME :
DATE: REG. NO. :

INPUT :

Enter the pass band ripple:0 .4
Enter the stop band ripple:30

SIMULATED OUTPUT:





RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

CHEBYSHEV LOW PASS FILTER

AIM :

To write a program in MATLAB to perform the chebyshev LPF operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Calculate the order of the filter using the formula,

   N = (cosh^-1(√((10^0.1αs-1)/(10^0.1αp-1))))/(cosh^-1(Ωs/Ωp)).

3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

FORMULA:

Chebyshev analog filter is defined via expression:

$$\left|H_a(\Omega)\right|^2 = H_a(j\Omega)\cdot H_a(-j\Omega) = \frac{1}{1+\varepsilon^2\cdot T_N{}^2(\Omega)}$$

Ω is the frequency;
N is the filter order;
ε is a parameter used to define maximum oscillations in the passband frequency response; and
TN is the Chebyshev polynomial. The Chebyshev polynomial TN(Ω) can be obtained via recursive relations:

$$T_0(x) = 1$$
$$T_1(x) = x$$
$$T_{n+1}(x) = 2\cdot x\cdot T_n(x) - T_{n-1}(x)$$

The values of poles are expressed as:

$$s_i = \sigma_i + j\Omega_i$$

Where,
si is the i-th transfer function pole of analog prototype filter (complex value);
σi is the pole; and Ωi is the imaginary pole.

$$\sigma_i = \sinh\left(\frac{1}{N}\cdot\sinh^{-1}\frac{1}{\varepsilon}\right)\cdot\cos\left(\frac{2\cdot i+N+1}{2\cdot N}\cdot\pi\right)$$

$$\Omega_i = \cosh\left(\frac{1}{N}\cdot\sinh^{-1}\frac{1}{\varepsilon}\right)\cdot\sin\left(\frac{2\cdot i+N+1}{2\cdot N}\cdot\pi\right)$$

Where,
N is the filter order; and i=1, 2, ..., N.
The value of parameter ε is obtained via expression:

$$\varepsilon = \sqrt{10^{\frac{a_p}{10}} - 1}$$

Transfer function is expressed as:

$$H_a(s) = \frac{A_o}{\prod_{k=1}^{N}(s - s_k)}$$

The value of A0 is found via expression:

$$A_0 = \begin{cases} 10^{-0.05a_p}\prod_{p=1}^{N}(-s_p) & for\,even\,N \\ \prod_{p=1}^{N}(-s_p) & for\,odd\,N \end{cases}$$

For N=5, the transfer function is:

$$H_a(s) = \frac{1}{(s-s_0)\cdot(s-s_1)\cdot(s-s_2)\cdot(s-s_3)\cdot(s-s_4)}$$

Low-pass filter:

In the transform function, **s\\Ωc** is used instead of **s**, where **Ωc** is a desirable cut-off frequency in the passband.
For high pass filter

$$s \rightarrow \frac{\Omega_c}{s}$$

The transformation in a band-stop analog filter:

$$s \rightarrow \frac{s}{s^2 + \Omega_0^2}$$

The value of the constant Ω0 can be found via expression:

$$\Omega_0 = \sqrt{\Omega_{s1}\Omega_{s2}}$$

s_1 is a lower cut-off frequency in the stopband and Ωs_2 is a higher cut-off frequency in the stopband.The transformation into a band-pass analog filter is expressed as:

$$s \rightarrow \frac{s^2 + \Omega_0^2}{s}$$

The value of the constant Ω0 can be found via expression:

$$\Omega_0 = \sqrt{\Omega_{p1}\Omega_{p2}}$$

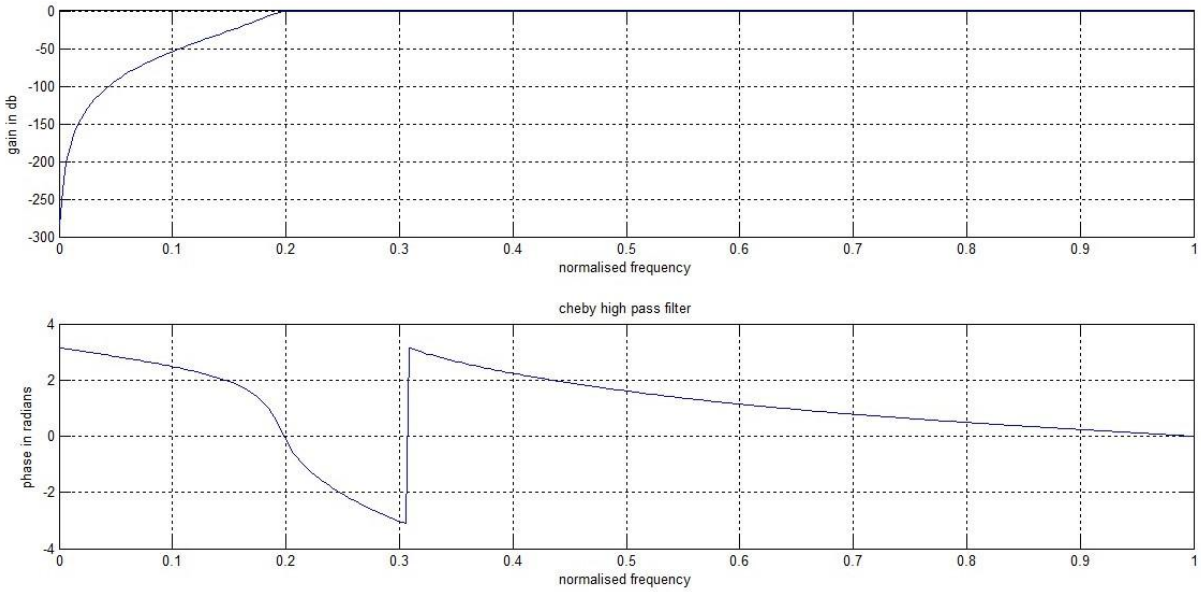
Where,Ωp1 is a lower cut-off frequency in the passband and Ωp2 is a higher cut-off frequency in the passband.

PROGRAM :

```
clc;
close all;
clear all;
ap=input('Enter the pass band attenuation in db');
as=input('Enter the Stop Band in db');

wp=.2*pi;
ws=.3*pi;
[n,wn]=cheb1ord(wp/pi,ws/pi,ap,as);
[b,a]=cheby1(n,ap,wn);
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain in db');
xlabel('Normalised Frequency');

subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase in Radians');
xlabel('Normalised Frequency');
title('Cheby Low Pass Filter');
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT:



INPUT :

Give the pass band attenuation in db : 0.4
Give the stop band in db : 30

RESULT :

Thus the program was executed in MATLAB and the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

CHEBYSHEV HIGH PASS FILTER

AIM :

To write a program in MATLAB to perform chebyshev HPF operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Calculate the order of the filter using the formula,

   N = (cosh^-1(√((10^0.1αs-1)/(10^0.1αp-1))))/(cosh^-1(Ωs/Ωp)).

3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

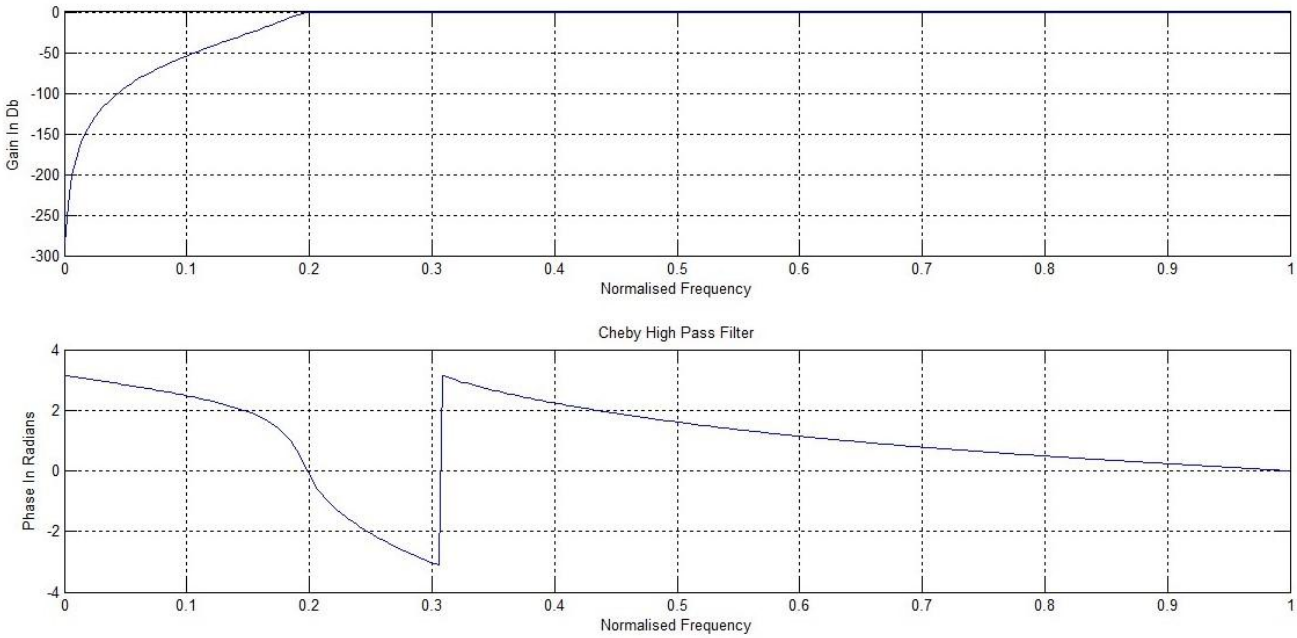
PROGRAM :

```
close all;
clear all;
clc;

ap=input('Enter The Pass Band Attenuation In Db');
as=input('Give The Stop Band In Db');
wp=.2*pi;
ws=.3*pi;
[n,wn]=cheb1ord(wp/pi,ws/pi,ap,as);cv
[b,a]=cheby1(n,ap,wn,'High');
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain In Db');
xlabel('Normalised Frequency');
subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
xlabel('Normalised Frequency');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
title('Cheby High Pass Filter');
```

SIMULATED OUTPUT:



INPUT :

Give the pass band attenuation in db:0.4
Give the stop band attenuation in db:30

RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

5.C.CHEBYSHEV BAND PASS FILTER

AIM :

To write a program in MATLAB to perform chebyshev BPF operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Calculate the order of the filter using the formula,

   N = (cosh^-1(√((10^0.1αs-1)/(10^0.1αp-1))))/(cosh^-1(Ωs/Ωp)).

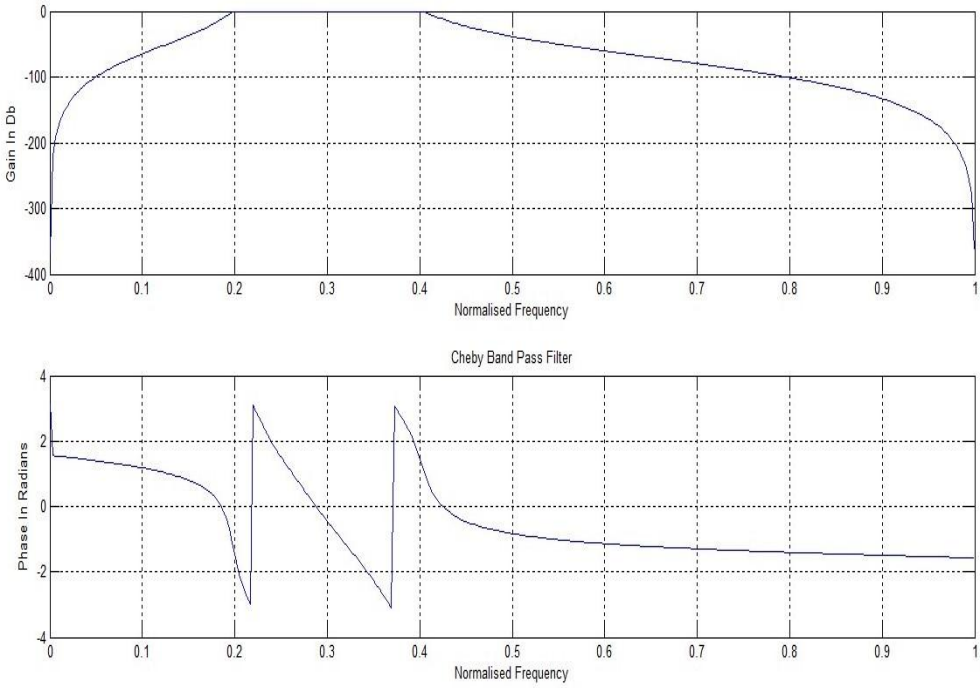
3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

PROGRAM :

```
clc;
close all;
clear all;
ap=input('Give The Pass Band Attenuation In Db');
as=input('Give The Stop Band In Db');
wp=[.2*pi,.4*pi];
ws=[.1*pi,.5*pi];
[n,wn]=cheb1ord(wp/pi,ws/pi,ap,as);
[b,a]=cheby1(n,ap,wn);
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain In Db');
xlabel('Normalised Frequency');
subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
xlabel('Normalised Frequency');
title('Cheby Band Pass Filter');
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT:



INPUT :

Give the pass band attenuation in db:0.4
Give the stop band in db :30

RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

5.D. CHEBYSHEV BAND STOP FILTER

AIM :

To write a program in MATLAB to perform chebyshev BSF operation.

ALGORITHM :

1. Get the pass band and stop band ripples.
2. Calculate the order of the filter using the formula,

   N = (cosh^-1(√((10^0.1αs-1)/(10^0.1αp-1))))/(cosh^-1(Ωs/Ωp)).

3. Find the filter coefficients.
4. Draw the magnitude and phase responses.
5. Stop.

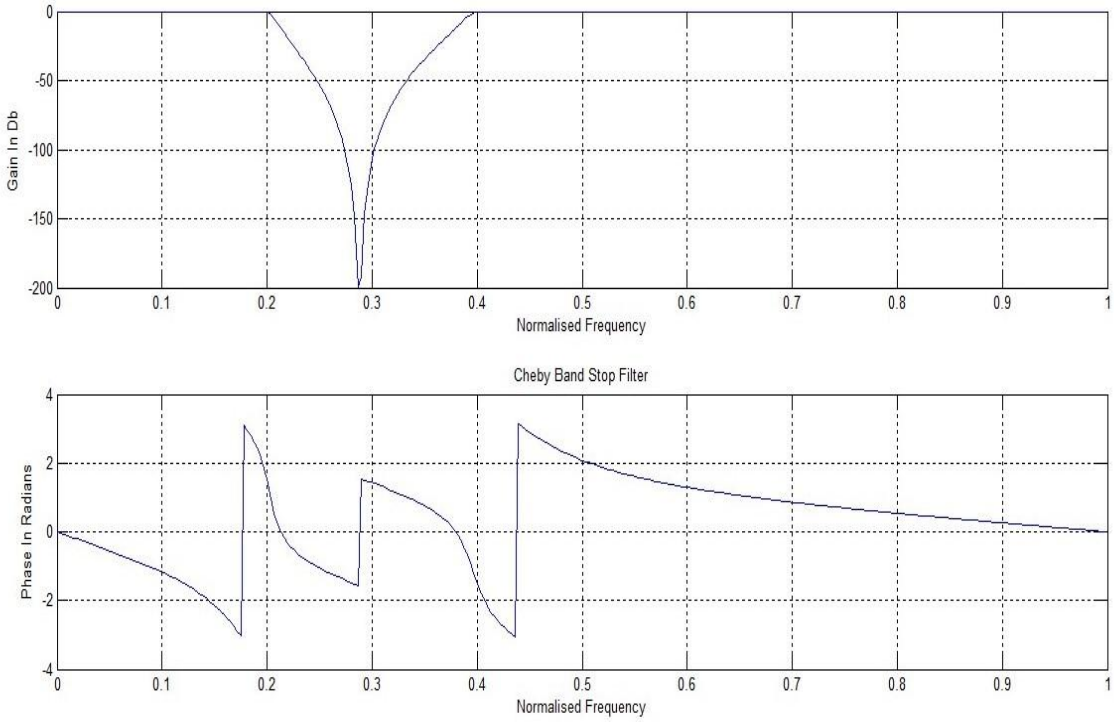
PROGRAM :

```
clc;
close all;
clear all;
ap=input('Give The Pass Band Attenuation In Db');
as=input('Give The Stop Band In Db');
wp=[.2*pi,.4*pi];
ws=[.1*pi,.5*pi];
[n,wn]=cheb1ord(wp/pi,ws/pi,ap,as);
[b,a]=cheby1(n,ap,wn,'Stop');
w=0:0.01:pi;
[h,om]=freqz(b,a,w);
m=20*log10(abs(h));
an=angle(h);
subplot(2,1,1);
plot(om/pi,m);
grid;
ylabel('Gain In Db');
xlabel('Normalised Frequency');
subplot(2,1,2);
plot(om/pi,an);
grid;
ylabel('Phase In Radians');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
xlabel('Normalised Frequency');
title('Cheby Band Stop Filter');
```

SIMULATED OUTPUT:



INPUT :

Give the pass band attenuation in db:0.4
Give the stop band in db:30

RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

6.A RECTANGULAR AND HAMMING WINDOW

AIM :

To write a program in MATLAB to perform rectangular & hamming window operation.

ALGORITHM :

1. Clear all the m-files &close them.
2. Get wc1 & wc2 values.
3. Calculate hd(n), hw(n), h(n) for various values for rectangular window.

$$hd(n)=1/2\pi\int_{-\infty}^{\infty} H(e^{j\omega}).e^{j\omega} \qquad h\omega(n)=1.$$

4. Calculate hd(n), w(n), h(n) for various values for hamming window.

$$w(n) = \begin{cases} 0.5 - 0.5\cos\frac{2\pi n}{N-1} & n = 0, 1, \ldots, N-1 \\ 0 & \text{otherwise} \end{cases}$$

$$hd(n)=1/\pi n \sin(\pi n/2).$$

$$h\omega(n) = \begin{cases} 0.54+0.46\cos(2\pi n/N-1) & ; -(N+1/2)\leq n\leq(N-1/2) \\ 0 & ; \text{otherwise.} \end{cases}$$

$$\begin{cases} I\circ(\beta)/I\circ(\alpha) & ; -(N+1/2)\leq n\leq(N-1/2) \\ 0 & ;\text{else.} \end{cases}$$

$$\beta = \alpha\,(1-(2n/N-1)^2)^{1/2}.$$

5.Plot the graph.
6.Stop.

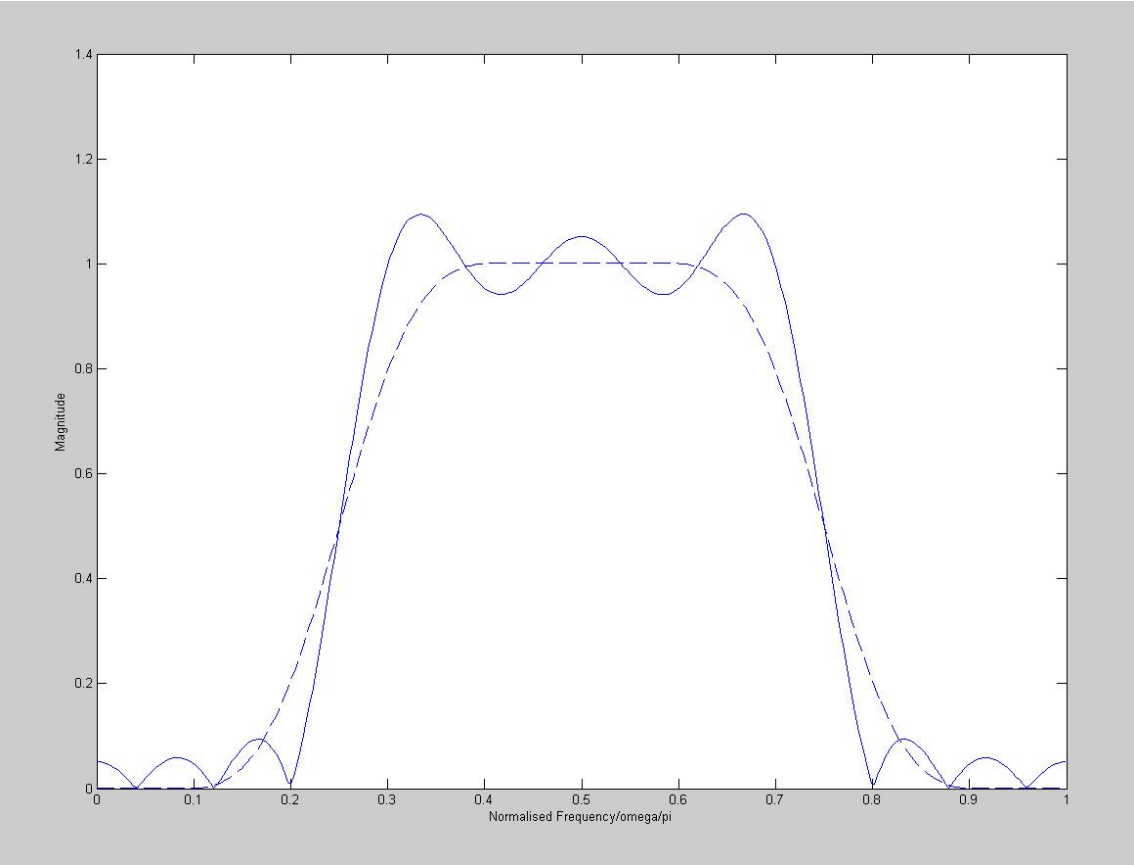
EX. NO. : NAME :
DATE: REG. NO. :

PROGRAM :

```
clc;
clear all;
close all;
wc1=.25*pi;
wc2=.75*pi;
N=25;
a=(N-1)/2;
eps=0.001;
n=0:1:N-1;
hd=(sin(wc2*(n-a+eps))-sin(wc1*(n-a+eps)))./(pi*(n-a+eps));
wr=boxcar(N);
hn=hd.*wr';
w=0:0.01:pi;
h=freqz(hn,1,w);
plot(w/pi,abs(h));hold on
wh=hamming(N);
hn=hd.*wh';
w=0:0.01:pi;
h=freqz(hn,1,w);
plot(w/pi,abs(h),'--');
xlabel('Normalised Frequency/omega/pi');
ylabel('Magnitude');
hold off;
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT:



RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

6.B KAISER WINDOW

AIM :

To write a program in MATLAB to perform kaiser window operation.

ALGORITHM :

1. Clear all the m-files & close them.
2. Calculate β and w(n)

$$\beta = \begin{cases} 0.1102(\alpha - 8.7), & \alpha > 50 \\ 0.5842(\alpha - 21)^{0.4} + 0.07886(\alpha - 21), & 50 \geq \alpha \geq 21 \\ 0, & \alpha < 21 \end{cases}$$

$$w(n) = \begin{cases} \dfrac{I_0\left(\beta\frac{\sqrt{n(N-1-n)}}{N-1}\right)}{I_0(\beta)} & n = 0, 1, \ldots, N-1 \\ 0 & \text{otherwise} \end{cases}$$

3.Find Wc = 0.5 * Pi
4.Calculate hd(n), hw(n), h(n) for various values of Kaiser window.
5.Plot the graph n

Hint:

To create the Kaiser windows, use the Matlab command kaiser(N,beta) command where N is the length of the filter and beta is the shape parameter β.

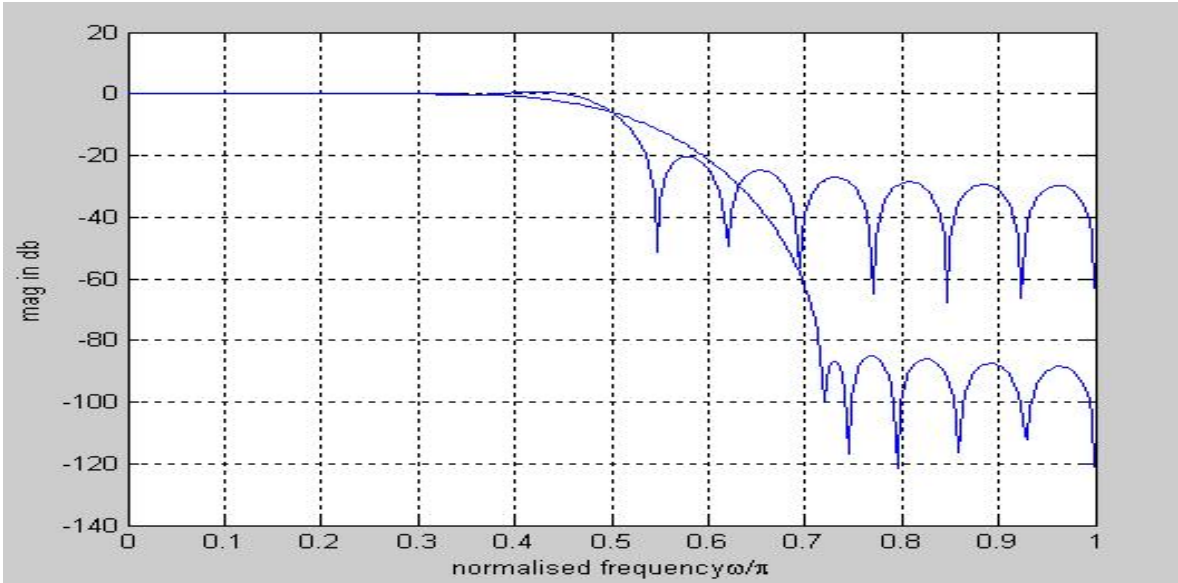
PROGRAM :

```
clc;
close all;
clear all;
wc=0.5*pi;
N=25;
b=fir1(N,wc/pi,kaiser(N+1,8.5));
w=0:0.01:pi;
h=freqz(b,1,w);
plot(w/pi,20*log10(abs(h)));
hold on;
b=fir1(N,wc/pi,kaiser(N+1,8.5));
w=0:0.01:pi;
```

EX. NO. : NAME :
DATE: REG. NO. :

```
h=freqz(b,1,w);
plot(w/pi,20*log10(abs(h)),'-');
grid;
hold on;
xlabel('Normalised Frequency/omega/pi');
ylabel('Magnitude In Db');
hold off;
```

SIMULATED OUTPUT:



RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

7.B PERIODIC & APERIODIC SEQUENCES

AIM:

To write program in MATLAB to generate periodic & aperiodic sequence.

ALGORITHM:

1. Define the values of n,n1,etc.
2. Split the given function for step of 1 (or) 0.1,etc
3. Calculate a signal that is periodic with respect to the input signal.
4. Plot the graph of the given input signal and periodic signal.
5. Calculate a signal that is a periodic with r to the input signal.
6. Plot the graph of given input signal and aperiodic signal.
7. Stop

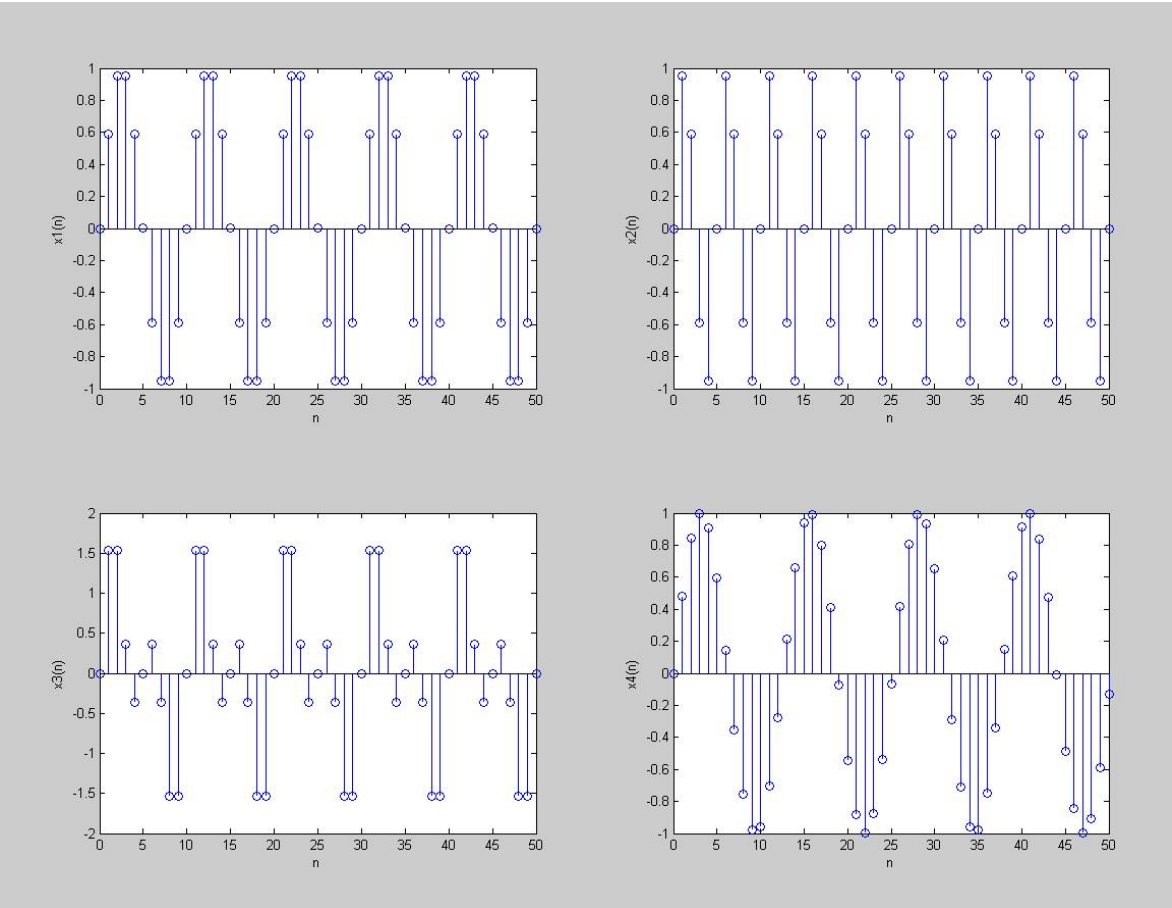
PROGRAM:

```
clear all;
n=0:1:50;
n1=0:1:50;
x1=sin(.2*pi*n);
x2=sin(.4*pi*n);
x3=sin(.2*pi*n)+sin(.4*pi*n);
x4=sin(.5*n1)
subplot(2,2,1);
stem(n,x1);
xlabel('n');
ylabel('x1(n)');
axis([0 50 -1 1]);
subplot(2,2,2);
stem(n,x2);
xlabel('n');
ylabel('x2(n)');
axis([0 50 -1 1]);
subplot(2,2,3);
stem(n,x3);
xlabel('n');
ylabel('x3(n)');
```

EX. NO. : NAME :
DATE: REG. NO. :

```
axis([0 50 -2 2]);
subplot(2,2,4);
stem(n1,x4);
xlabel('n');
ylabel('x4(n)');
axis([0 50 -1 1]);
```

SIMULATED OUTPUT:



RESULT :

Thus the program was executed in MATLAB & the output was verified.

EX. NO. : NAME :
DATE: REG. NO. :

9. Z TRANSFORMS AND INVERSE Z TRANSFORMS

A. Z TRANSFORM

AIM:

To write a program to illustrate the Z transforms process using MATLAB functions.

ALGORITHM:

1.Mention Number of Frequency Points, Numerator Coefficients and Denominator Coefficients.
2.Compute the frequency response of Z transform of Unit circle
3. Plot the Frequency Response.
4.Plot the Z transform using freqz function in MATLAB.
5. Plot the Real, Imaginary and Magnitude plots,

Formula:

The *z*-transform is defined as

$$X(z) = \sum_{k=0}^{N} x[k]z^{-k} = \sum_{k=0}^{N} x[k](z^{-1})^{k}$$

The inverse Z transform is given by

$$x[n] = \sum_{k=0}^{N} x[k]\delta[n-k] \overset{z}{\leftrightarrow} X(z) = \sum_{k=0}^{N} x[k]z^{-k}$$

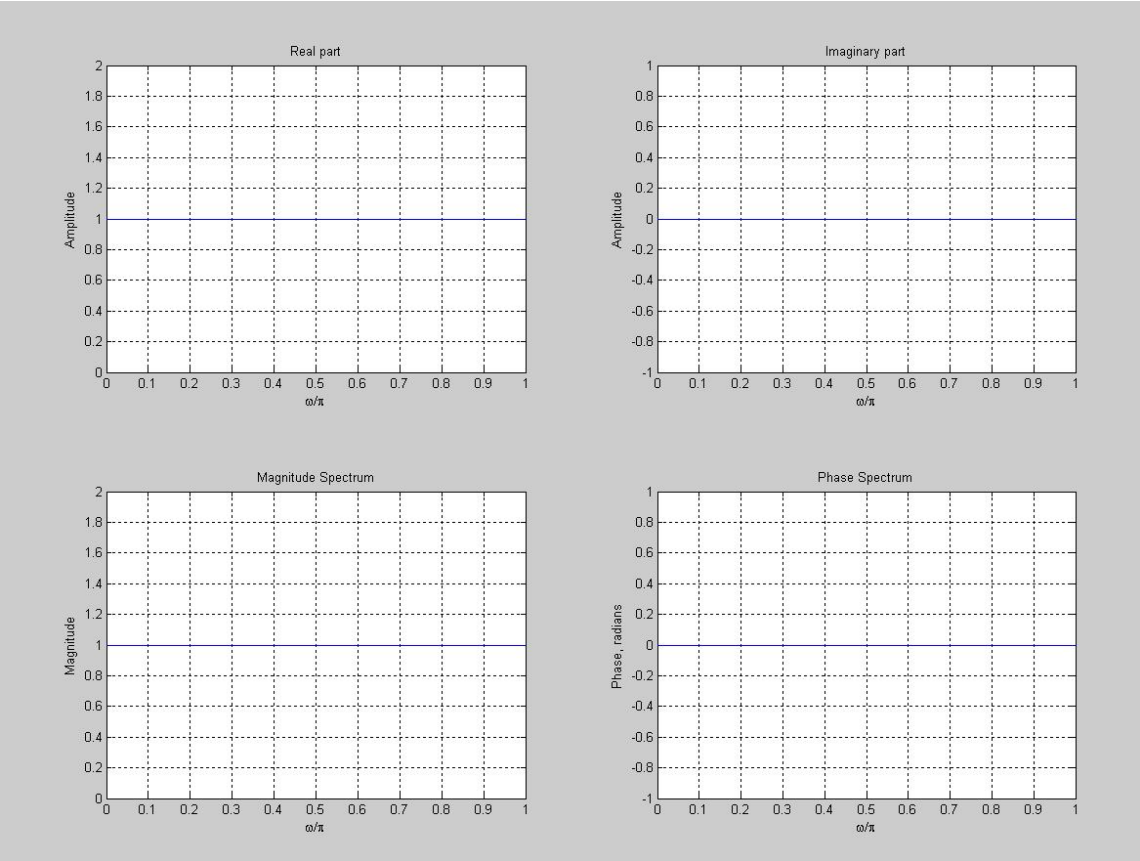
PROGRAM:

```
close all;
clear all;
clc;
k = input('Number of frequency points = ');
% Read in the numerator and denominator coefficients
num = input('Numerator coefficients = ');
den = input('Denominator coefficients = ');
% Compute the frequency response/Evaluate Z transform on unit circle
w = 0:pi/(k-1):pi;
h = freqz(num, den, w);
% Plot the frequency response
subplot(2,2,1)
```

EX. NO. : NAME :
DATE: REG. NO. :

```
plot(w/pi,real(h));grid
title('Real part')
xlabel('\omega/\pi'); ylabel('Amplitude')
subplot(2,2,2)
plot(w/pi,imag(h));grid
title('Imaginary part')
xlabel('\omega/\pi'); ylabel('Amplitude')
subplot(2,2,3)
plot(w/pi,abs(h));grid
title('Magnitude Spectrum')
xlabel('\omega/\pi'); ylabel('Magnitude')
subplot(2,2,4)
plot(w/pi,angle(h));grid
title('Phase Spectrum')
xlabel('\omega/\pi'); ylabel('Phase, radians')
```

SIMULATED OUTPUT:



RESULT: The program is executed successfully and plotted input and output waveforms.

EX. NO. : NAME :
DATE: REG. NO. :

9.B. INVERSE Z TRANSFORMS

AIM:

To write a program to illustrate the Inverse Z transforms process using MATLAB functions.

ALGORITHM:

1. The function impz provides the samples of the time domain sequence

2. where num and den are row vectors containing the coefficients of the numerator and denominator polynomials of $G(z)$ in ascending powers of $z-1$, L is the desired number of the samples of the inverse transform,

3. g is the vector containing the samples of the inverse transform starting with the sample at $n = 0$, t is the length of g, and FT is the specified sampling frequency in Hz with default value of unity.

4. The function y=filter(num,den,x) can also be used to calculate the input response of a z-transform . Where num, den represent vectors containing numerator and denominator coefficients of z-transform.

5. While x represents input to the filter / z-transform .The length of output y is equal to input x. If an impulse input sequence is passed to the z-transform , the output will be the inverse z-transform

6. Plot the Frequency Response.

7. Plot the Z transform using freqz function in MATLAB.

8. Plot the Real, Imaginary , Magnitude and Phase plots.

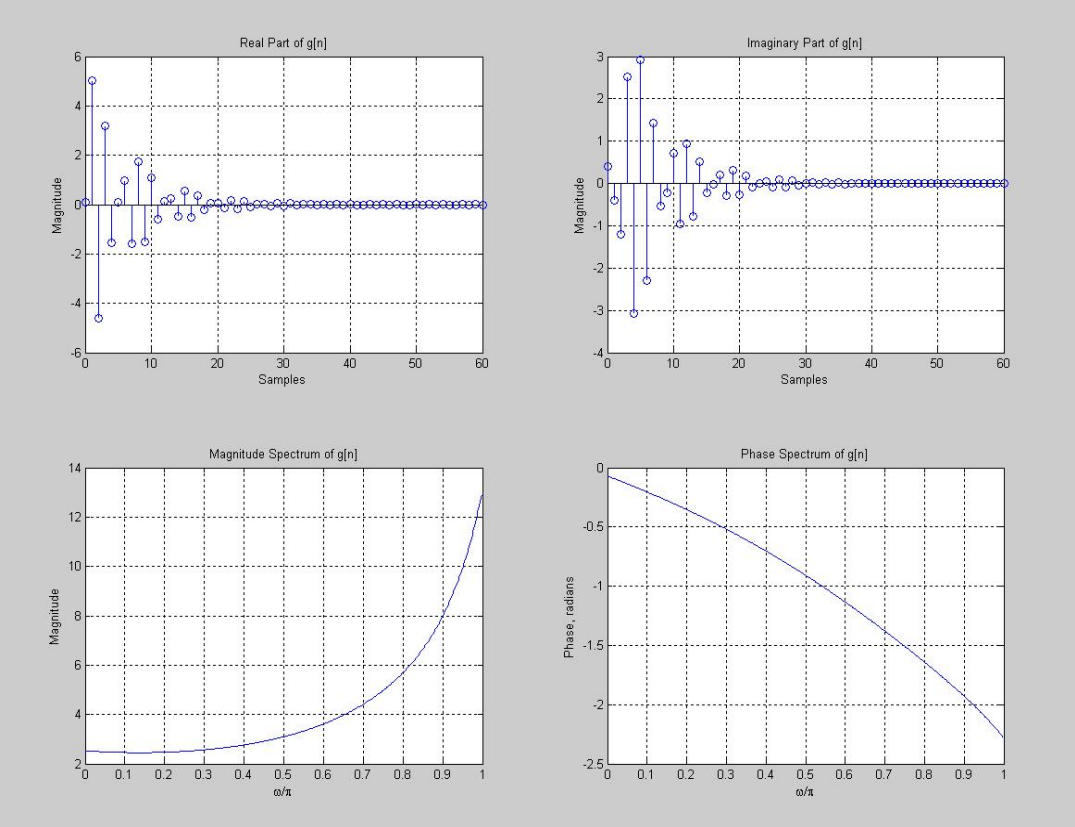
EX. NO. : NAME :
DATE: REG. NO. :

PROGRAM:

```
close all;
clear all;
clc;
%definition of numerator and denominator coefficients
num=[0.1+.4*i 5 .05];
den=[1 .9+0.3*i .12];
%Finding the inverse z transform of G(z)
[a,b]=impz(num,den);
%Evaluating on Unit Circle i.e. Fourier Transform
[c,d]=freqz(num,den);
% Plotting of x[n] and it's fourier transform
subplot(2,2,1)
stem(b,real(a))
title('Real Part of g[n]')
xlabel('Samples'); ylabel('Magnitude')
grid on
subplot(2,2,2)
stem(b,imag(a))
title('Imaginary Part of g[n]')
xlabel('Samples'); ylabel('Magnitude')
grid on
subplot(2,2,3)
plot(d/pi,abs(c))
title('Magnitude Spectrum of g[n]')
xlabel('\omega/\pi'); ylabel('Magnitude')
grid on
subplot(2,2,4)
plot(d/pi,angle(c))
title('Phase Spectrum of g[n]')
xlabel('\omega/\pi'); ylabel('Phase, radians')
grid on
```

EX. NO. : NAME :
DATE: REG. NO. :

SIMULATED OUTPUT:



Z TRANSFORM OUTPUT:

Number of frequency points = 3
Numerator coefficients = 3
Denominator coefficients = 3

RESULT:

The program is executed successfully and plotted input and output waveforms.

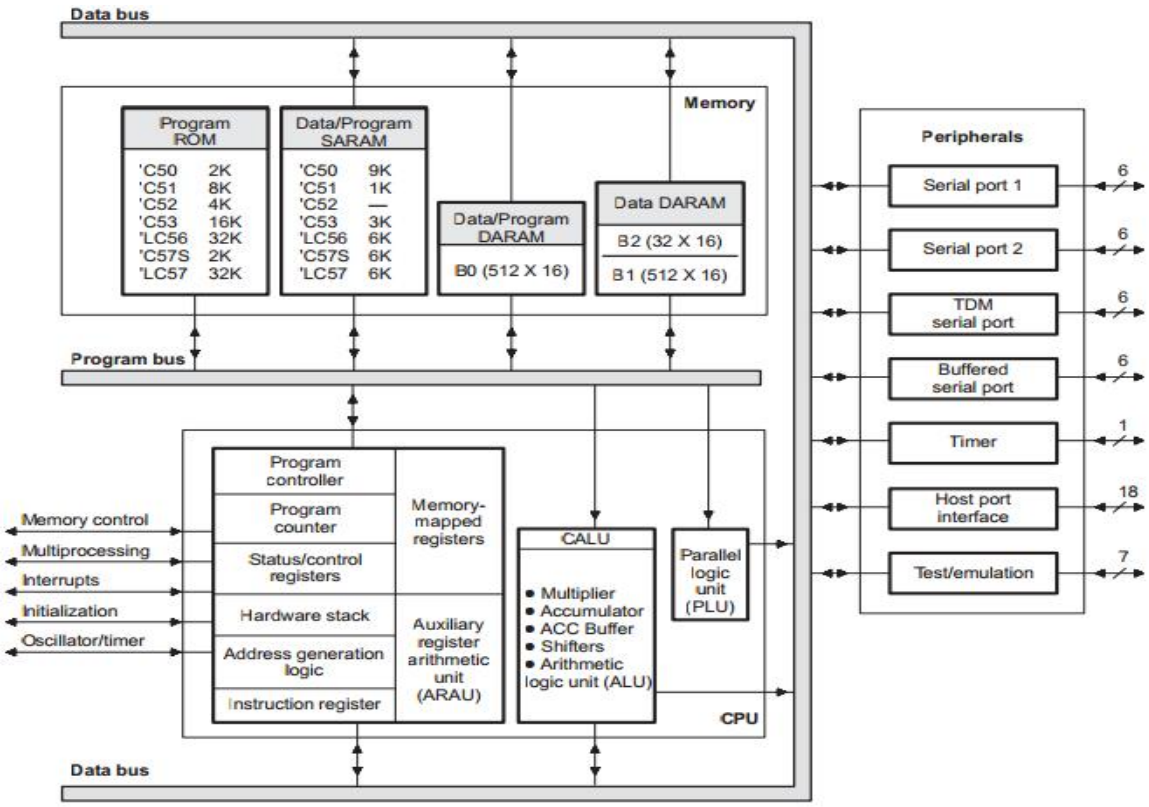
EX. NO. : NAME :
DATE: REG. NO. :

10. A. STUDY OF TMS320C3X PROCESSOR

AIM:

These processors can perform parallel multiply and arithmetic logic unit (ALU) operations on integer or floating-point data in a single cycle.

FUNCTIONAL BLOCK DIAGRAM:



EXPLANATION:

- Separate program and data buses allow simultaneous access to program instructions and data, providing a high degree of parallelism. Such parallelism supports a powerful set of arithmetic, logic, and bit-manipulation operations that can all be performed in a single machine cycle. It also includes the control mechanisms to manage interrupts ,repeated operations, and function calling.

- The ’C5x architecture is built around four major buses:
  - Program bus (PB): It carries the instruction code and immediate operands from program memory space to the CPU.

  - Program address bus (PAB) :It provides addresses to program memory space for both reads and writes

- Data read bus (DB) : It interconnects various elements of the CPU to data memory space.
- Data read address bus :It provides the address to access the data memory space.

- The ’C5x CPU consists of these elements:
  - Central arithmetic logic unit (CALU)
  - Parallel logic unit (PLU)
  - Auxiliary register arithmetic unit (ARAU)
  - Memory-mapped registers
  - Program controller
- The CPU uses the CALU to perform 2scomplement arithmetic. The CALU consists of these elements:
  - 16-bit multiplier
  - 32-bit arithmetic logic unit (ALU)
  - 32-bit accumulator (ACC)
  - 32-bit accumulator buffer (ACCB)
  - Additional shifters at the outputs of both the accumulator and the product register (PREG)
- The CPU includes an independent PLU, which operates separately from, but in parallel with, the ALU. The PLU performs Boolean operations or the bit manipulations required of high-speed controllers.
- The PLU can set, clear, test, or toggle bits in a status register, control register, or any data memory location.
- Results of a PLU function are written back to the original data memory location.
- The CPU includes an unsigned 16-bit arithmetic logic unit that calculates indirect addresses by using inputs from the auxiliary registers (ARs), index register (INDX), and auxiliary register compare register (ARCR).
- Accessing data does not require the CALU for address manipulation; therefore, the CALU is free for other operations in parallel. This makes instruction to executes faster compared to micro processors.

- The program controller contains logic circuitry that decodes the operational instructions, manages the CPU pipeline, stores the status of CPU operations,and decodes the conditional operations.
  - The program controller consists of these elements:
  - Program counter
  - Status and control registers
  - Hardware stack
  - Address generation logic
  - Instruction register
- ’C5x architecture contains a considerable amount of on-chip memory to aid in system performance and integration:
  - Program read-only memory (ROM)
  - Data/program dual-access RAM (DARAM)
  - Data/program single-access RAM (SARAM
- **Program ROM:**
  - All c5x dsp carry 16 bit programmable ROM.
  - It is used for booting program from slower external RAM to faster external RAM.
- **DARAM:** (Dual access RAM)
  - All c5x dsp carry 1056 word x 16 bit on chip DARAM.
  - It is divided into three blocks
  - 512 word/pgm DARAM block B0.
  - 512 word/pgm DARAM block B1.
  - 32 word data DARAM block B2.
  - Block B1,B2 configured as data memory and B0 configured software as data or program memory.
- **SARAM:** (Single access RAM)
  - All c5x dsp carry 16 bit on chip single access RAM.

- It is configured by software as data memory, as program memory and both combination of both data and program.

- **Clock Generator:**The clock generator consists of an internal oscillator and a phase-locked loop(PLL) circuit. The clock generator can be driven internally by a crystal resonator. The PLL circuit can generate an internal CPU clock by multiplying the clock source by a specific factor.

- **Hardware Timer:**A 16-bit hardware timer with a 4-bit prescaler is available. The timer can be stopped, restarted, reset, or disabled by specific status bits.

- **Software-Programmable Wait-State Generators:** Software-programmable wait-state logic is incorporated in ’C5x DSPs allowing wait-state generation without any external hardware . This feature consists of multiple wait state generating circuits. Each circuit is user-programmable to operate in different wait states for off-chip memory accesses.

- **Host Port Interface (HPI) :**The HPI available on the ’C57S and ’LC57 is an 8-bit parallel I/O port that provides an interface to a host processor. Information is exchanged between the DSP and the host processor through on-chip memory that is accessible to both the host processor and the ’C57.

- **Serial Port :** Three different kinds

- **a general-purpose serial port**

- ***(TDM) serial port*** : The TDM serial port available on the ’C50, ’C51, and ’C53 devices is a fullduplexed

- serial port that can be configured by software either for synchronous

- operations or for time-division multiplexed operations. The TDM serial port is

- commonly used in multiprocessor applications

- **A buffered serial port (BSP) :** The BSP available on the ’C56 and ’C57 devices is a full-duplexed, double buffered serial port and an autobuffering unit (ABU). The BSP provides flexibility on the data stream length.

- The serial port transmitter and receiver are double-buffered and individually controlled by maskable external interrupt signals. Data is framed either as bytes or as words.

RESULT:

Studied the architecture and explanation of the processor.

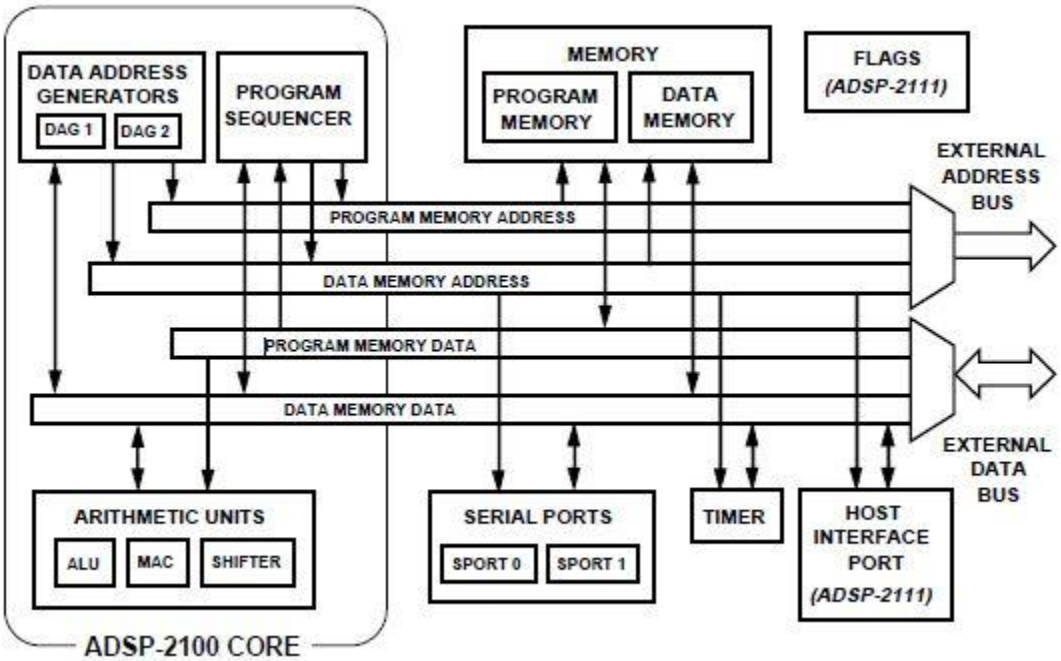
EX. NO. : NAME :
DATE: REG. NO. :

10.B. STUDY OF ADSP21XX PROCESSOR

AIM:

The ADSP-2100 Family processors are single-chip microcomputers optimized for digital signal processing (DSP) and other high speed numeric processing applications

ARCHITECTURE:



EXPLANATION:

1. 16-Bit Fixed-Point DSP Microprocessors with On-Chip Memory Enhanced Harvard Architecture for Three-Bus Performance: Instruction Bus & Dual Data Buses Independent Computation Units: ALU, Multiplier/ Accumulator, and Shifter Single-Cycle Instruction Execution & Multifunction Instructions On-Chip
2. Program Memory RAM or ROM & Data Memory RAM Integrated I/O Peripherals: Serial Ports, Timer, Host Interface Port (ADSP-2111 Only)
3. 25 MIPS, 40 ns Maximum Instruction Rate Separate On-Chip Buses for Program and Data Memory Program Memory Stores Both Instructions and Data (Three-Bus Performance) Dual Data Address Generators with Modulo and Bit-Reverse Addressing Efficient Program Sequencing with Zero-Overhead Looping: Single-Cycle Loop Setup Automatic Booting of On-Chip Program Memory from Byte-

Wide External Memory (e.g., EPROM) Double-Buffered Serial Ports with Companding Hardware, Automatic Data Buffering, and Multichannel Operation ADSP-2111 Host Interface Port Provides Easy Interface to 68000, 80C51, ADSP-21xx, Etc. Automatic Booting of ADSP-2111 Program Memory Through Host Interface Port Three Edge- or Level-Sensitive Interrupts Low Power IDLE Instruction PGA, PLCC, PQFP, and TQFP Packages MIL-STD-883B Versions Available

4. The ADSP-2100 Family processors are single-chip microcomputers optimized for digital signal processing (DSP) and other high speed numeric processing applications. The ADSP-21xx processors are all built upon a common core. Each processor combines the core DSP architecture—computation units, data address generators, and program sequencer—with differentiating features such as on-chip program and data memory RAM, a programmable timer, one or two serial ports, and, on the ADSP-2111, a host interface port.
5. Fabricated in a high speed, submicron, double-layer metal CMOS process, the highest-performance ADSP-21xx processors operate at 25 MHz with a 40 ns instruction cycle time. Every instruction can execute in a single cycle. Fabrication in CMOS results in low power dissipation.
6. The ADSP-2100 Family's flexible architecture and comprehensive instruction set support a high degree of parallelism. In one cycle the ADSP-21xx can perform all of the following operations.

RESULT:

Studied Architecture and explanation of the ADSP21xx Processor.

EX. NO. : NAME :
DATE: REG. NO. :